

Hướng dẫn đọc code — Ảnh xạ lý thuyết ↔ Code

Tài liệu này đi từng dòng luồng chính của mã nguồn, đối chiếu với lý thuyết (Mamba/SSM, Conformer, dual-path, SI-SNR, F0 conditioning) để bạn đọc hiểu "vì sao code viết như vậy". Đọc theo thứ tự mục 3 → 4.1 → 4.3 → 4.4 → 4.7 → 4.10 là đủ để nắm toàn bộ. Các đoạn code trích nguyên văn từ repository.

1. Cách dùng tài liệu này

Ký hiệu	Ý nghĩa
Lý thuyết	Công thức / ý tưởng khoa học
Code	Đoạn code thật (đường dẫn file → hàm)
Ảnh xạ	Nối lý thuyết với dòng code tương ứng

Đọc một module = đọc 3 ô trên của mục module đó.

2. Bản đồ tổng thể: Lý thuyết → File

Khái niệm lý thuyết	File / class / hàm chính
Selective State Space (S6/Mamba-1): $h_t = \tilde{A}h_{t-1} + B\tilde{u}_t, y_t = Ch_t + Du_t$	<code>`conformer_bimamba/models/ssm.py`</code> → <code>F0ConditionedSSM</code> , <code>`selective_scan_ref`</code>
Điều biến B, C theo F0 (thanh điệu tiếng Việt)	<code>`ssm.py`</code> → <code>`_modulate`</code> · <code>`models/f0_conditioning.py`</code> → <code>F0Projector</code>
Bidirectional Mamba (hai chiều, phi nhân quả offline)	<code>`models/bimamba.py`</code> → <code>BiMambaLayer</code>
Conformer block: <code>`FFN`</code> → <code>`MHSA`</code> → <code>`DWConv`</code> → <code>`FFN`</code> + half-step residual	<code>`models/hybrid_block.py`</code> → <code>ConformerBlock</code> / <code>HybridBlock</code> / <code>PreNorm</code>
DepthwiseConv bias cục bộ <30 ms	<code>`hybrid_block.py`</code> → <code>DepthwiseConv1d</code>
Dual-path (DPRNN/Sepformer): chunk intra/inter + overlap-add	<code>`models/separation.py`</code> → <code>DualPathProcessor</code> (+ <code>`_chunk`</code> , <code>`_overlap_add`</code>)
Positional encoding hình sin (bù cho Mamba không có vị trí tường minh)	<code>`models/positional.py`</code> → <code>SinusoidalPositionalEncoding</code>
Encoder/Decoder học (learned basis) + mask	<code>`separation.py`</code> → <code>Encoder</code> , <code>Decoder</code> , <code>HybridSepformer</code>
PIT SI-SNR loss (hoán vị 2 người nói)	<code>`utils/metrics.py`</code> → <code>si_snr</code> , <code>si_snr_pit</code> , <code>si_snr_loss</code>
Dynamic mixing (WHAM-style)	<code>`data/vivos.py`</code> → <code>SpeechSeparationDataset</code>
F0 offline (PyWorld dio + stonemask, 10 ms)	<code>`preprocess/extract_f0.py`</code> → <code>extract_f0_from_wav</code>
Mixture cố định val/test	<code>`preprocess/make_mixtures.py`</code> → <code>make_fixed_mixtures</code>
Huấn luyện: warmup+cosine, AMP, per-group LR, early-stop	<code>`train.py`</code> , <code>`utils/lr_scheduler.py`</code> , <code>`utils/seed.py`</code>
Bản ClearerVoice-Studio (MOSSFormer2-style)	<code>`train/speech_separation/`</code> (xem mục 4.11)

3. Luồng dữ liệu end-to-end

```
wav 16 kHz —load/resample 32 kHz→ dynamic mix 2 người (data/vivos.py)
|                                     mix, s1, s2 (mỗi cái 1×T), f0_mix (max của 2 source F0)
▼
train.py: mỗi step
  mixture(B,1,T) → model → est (B,2,1,T)      f0(B,L) → (điều kiện hóa)
  loss = -mean(PIT SI-SNR(est, refs))          (utils/metrics.py)
```

```
loss.backward() → clip → AdamW (mamba group LR×0.5) → warmup+cosine
▼ val mỗi 2 epoch: SI-SNRI/SDRi trên mixture cố định (mixtures/val)
▼ checkpoint: runs/<run>/checkpoints/{best,last}.pt (kèm config)
```

evaluate.py: test mixtures cố định → SI-SNRI, SDRi, PESQ, STOI, latency
inference.py: 1 wav → trích F0 (PyWorld) → model → s1.wav, s2.wav

4. Đi sâu từng module

4.1 `models/ssm.py` — Lỗi Selective State Space (Mamba-1) + F0 gate

Lý thuyết (S4/S6). Mamba là SSM rời rạc hóa như thuộc đầu vào:

```
x̃ = SiLU(DepthwiseConv1d(x)) # trộn cục bộ (d_conv=4) trước khi quét
Δ = softplus(Δ_proj(x) + Δ_bias) # bước thời gian (input-dependent)
A = -exp(A_log) # (d_inner, d_state)
B = B_proj(x), C = C_proj(x̃) # cổng vào/ra – phụ thuộc đầu vào
Ā_t = exp(Δ_t·A), B~_t = Δ_t·B_t # rời rạc hóa (ZOH)
h_t = Ā_t·h_{t-1} + B~_t·u_t # vòng lặp trạng thái – O(n)
y_t = C_t·h_t + D·u_t; y = y·SiLU(z) # cổng đầu ra (gated)
out = out_proj(y)
```

Code — `F0ConditionedSSM.forward` (trích nguyên văn):

```
xz = self.in_proj(x) # (1)
x_inner, z = xz.split([self.d_inner, self.d_inner], dim=-1)
x_inner = x_inner.transpose(1, 2)
x_inner = self.act(self.conv1d(x_inner)[ :, :, :L]) # (2) depthwise causal conv
x_dbl = self.x_proj(x_inner.transpose(1, 2))
dt, Bmat, Cmat = x_dbl.split([self.dt_rank, self.d_state, self.d_state], dim=-1)
dt = self.dt_proj.weight @ dt.transpose(1, 2) # (3) (B, d_inner, L)
if f0 is not None and self.f0_gate is not None:
    Bmat, Cmat = self._modulate(Bmat, Cmat, f0) # (4) F0 gate trên B, C
A = -torch.exp(self.A_log) # (5) A = -exp(A_log)
... B_scan/C_scan/z_scan = ...transpose(1, 2)
y = selective_scan_fn(u, dt, A, B_scan, C_scan, self.D, z_scan,
    delta_bias=..., delta_softplus=True) # (6) kernel CUDA
y = y.transpose(1, 2) * self.act(z) # (7) y·SiLU(z)
return self.out_proj(y) # (8) (B, L, d_model)
```

Ảnh xạ từng dòng:

- (1) `in_proj` chỉ chiếu $[x, z]$ (khớp **đúng** bố cục Mamba-1 gốc — dt, B, C không nằm trong `in_proj`); `d_inner = expand.d_model`.
- (2) conv depthwise **nhân qua** (padding `d_conv=1` rồi cắt bỏ phần pad cuối `[ :, :, :L]` — chính là bước "trộn cục bộ" trước quét.
- (3) dt, B, C lấy từ x *sau conv* (theo Mamba gốc); dt chiếu qua `dt_proj.weight` (bias được kernel cộng sau qua `delta_bias`).
- (4) **Điểm đóng góp của dự án:** $B = B \cdot (1 + \tanh(\text{gate}_B))$, $C = C \cdot (1 + \tanh(\text{gate}_C))$ — xem 4.2 (F0). Vì `selective_scan_fn` nhận B, C **tương minh** nên chèn được; kernel hợp nhất Mamba-2 thì không → lý do use `mamba2` chỉ dùng khi không F0.
- (5) `A_log` khởi tạo `log(arange(1, d_state+1))` (S4D real init), dùng dưới dạng `buffer` (không train) — A cố định như Mamba gốc.
- (6) Kernel CUDA (`mamba_ssm.ops.selective_scan_interface.selective_scan_fn`); khi không có (macOS/CI) chạy `selective_scan_ref` — **vòng lặp Python theo t** đúng công thức lý thuyết từng bước (xem hàm `selective_scan_ref` cùng file), dùng để kiểm thử/suy luận nhỏ.
- (7) cổng SiLU nhân vào đầu ra (gated output của Mamba); (8) chiếu về `d_model`.

Vì sao khởi tạo `dt` kiểu đó? `dt_proj.bias = exp(U(log dt_min, log dt_max)) → softplus(bias) ∈ [dt_min, dt_max]` ngay từ đầu (tránh quét nổ/chết), và `dt_proj.weight ~ U(-r^-0.5, r^-0.5)` (giữ phương sai) — đúng init Mamba gốc.

4.2 `models/bimamba.py` — Bidirectional Mamba

Lý thuyết. Mamba thuần nhân quả (chỉ nhìn quá khứ). Tách giọng offline được phép nhìn toàn bộ câu → chạy 2 chiều: thuận (left→right) + nghịch (right→left, bằng cách quét chuỗi đảo), rồi kết hợp. Mỗi chiều vẫn nhân quả *trong định hướng của nó* ⇒ không trộn quá khứ/tương lai xuyên chiều ⇒ không rò rỉ thông tin tương lai (offline separation hợp lệ, chuẩn WSJ0-2mix).

Code — `BiMambaLaver.forward`:

```
h_fwd = self.fwd(x, f0)
x_rev = torch.flip(x, dims=[1]) # đảo chuỗi
```

```
h_bwd = torch.flip(self.bwd(x_rev, f0_rev), dims=[1]) # quét ngược rồi đảo lại
return h_fwd + h_bwd # combine="sum" (hoặc concat+Linear)
```

Ảnh xạ: fwd/bwd là hai bản F0ConditionedSSM riêng (không chia sẻ trọng số) — mỗi chiều học biểu diễn riêng. combine="concat" nối 2 chiều rồi chiếu Linear($2 \cdot d \rightarrow d$). Tham số F0 gate tính 2 lần (2 chiều) — đã ghi trong parameter_analysis.md.

4.3 models/hybrid_block.py — Khối Conformer & khối Hybrid

Lý thuyết (Conformer, Gulati 2020). Mỗi khối: FFN $\rightarrow$ MHSA $\rightarrow$ DWConv $\rightarrow$ FFN với **half-step residual** (cộng $\frac{1}{2}$ FFN ở đầu/cuối) và **PreNorm** (LayerNorm trước hàm, residual sau). Bản hybrid chỉ đổi MHSA $\rightarrow$ BiMamba, giữ nguyên FFN và DWConv để ablation sạch.

Code — HybridBlock forward (nguyên văn):

```
class PreNorm(nn.Module):
    def forward(self, x, *args, **kwargs):
        return self.fn(self.norm(x), *args, **kwargs) # LayerNorm TRƯỚC fn

class HybridBlock(nn.Module):
    def forward(self, x, f0=None):
        x = x + 0.5 * self.ffn1(x) # half-step FFN (pre-norm bên trong)
        x = x + self.mamba(x, f0) # Bi-Mamba thay MHSA ← ĐIỂM THAY THẾ
        x = x + self.conv(x) # DepthwiseConv1d (k=31, giữ nguyên)
        x = x + 0.5 * self.ffn2(x)
        return x
```

- ConformerBlock.forward giống hệt, chỉ dòng giữa là self.attn(x) (MHSA) $\rightarrow$

chênh lệch hiệu năng giữa 2 khối quy về đúng phép thay thế.

- FeedForward: Linear($d \rightarrow 4d$) $\rightarrow$ SiLU $\rightarrow$ Dropout $\rightarrow$ Linear($4d \rightarrow d$) (Conformer FFN).
- DepthwiseConv1d: Conv1d($d, d, k=31, groups=d, pad=15$) ("same") $\rightarrow$

BatchNorm1d $\rightarrow$ SiLU. Kích thước cửa sổ: tại 32 kHz với encoder stride 8, 31 taps $\approx 7,8$ ms — bias cực nhỏ <30 ms cho âm vị/phụ âm (giữ nguyên theo đề bài).

- PreNorm+residual đúng hợp đồng "đầu ra Bi-Mamba cộng residual rồi chuẩn hóa

trước DepthwiseConv" (LayerNorm của PreNorm(conv) nằm ngay sau residual Bi-Mamba).

4.4 models/separation.py — Dual-path + mask + encoder/decoder

Lý thuyết dual-path (DPRNN/Sepformer). Tín hiệu dài không đưa cả chuỗi vào một lần: cắt thành các đoạn chồng lấn (chunk) $\rightarrow$ mô hình hóa **trong đoạn** (intra, ngữ cảnh ngắn) rồi **liên đoạn** (inter, ngữ cảnh dài) $\rightarrow$ ghép lại bằng overlap-add. Số chunk: $C = (T' - S) / \text{hop} + 1$.

Code — DualPathProcessor forward (nguyên văn, đã rút gọn chú thích):

```
x, pad = self._pad(x) # pad để (T_pad - S) % hop == 0
chunks = x.unfold(-1, self.chunk_size, self.hop) # (B, N, C, S) (1)
chunks = chunks.permute(0, 2, 3, 1) # (B, C, S, N)
chunks = chunks + self.pe_intra.pe[:, :S].unsqueeze(1) # (2) PE chiều S
h = chunks.reshape(B * C, S, N) # gộp (B·C) làm "batch"
h = self._apply_blocks(self.intra_blocks, h, h_f0) # (3) intra
chunks = h.view(B, C, S, N).transpose(1, 2) # (B, S, C, N)
chunks = chunks + self.pe_inter.pe[:, :C].unsqueeze(0) # PE chiều C
h = chunks.reshape(B * S, C, N) # (4) inter (nhìn C = vị trí đoạn)
h = self._apply_blocks(self.inter_blocks, h, h_f0)
chunks = h.view(B, S, C, N).transpose(1, 2) # về (B, C, S, N)
out = self._overlap_add(chunks) # (5) overlap-add cửa sổ tam giác
return out[:, :, :T]
```

Ảnh xạ:

- (1) unfold tạo các đoạn chồng lấn: hop = chunk/2 $\rightarrow$ mỗi mẫu phủ 2 đoạn; pad

tính sao cho phủ chẵn số đoạn.

- (2) PE hình sin cộng trên chiều S (trong đoạn) và chiều C (liên đoạn) — vì

Mamba không có cơ chế vị trí tường minh (xem 4.6).

- (3) intra: mỗi "chuỗi" dài $S=250$ được xử lý bởi chồng HybridBlock

(tổng $B \cdot C$ chuỗi song song). F0 theo từng vị trí ($B \cdot C, S, f0$ dim).

- (4) inter: chuyển vị $\rightarrow$ mỗi chuỗi dài C (số đoạn ≈ 159 cho câu 5 s) xử lý bởi

cùng loại khối — đây là chỗ Mamba phát huy phụ thuộc xa tuyến tính thay cho attention bậc hai.

F0 dùng **tóm tắt mỗi đoạn** (mean theo S) broadcast.

- (5) overlap-add: nhân cửa sổ tam giác (tăng 0 $\rightarrow$ 1 trong hop, giảm 1 $\rightarrow$ 0 trong

hop) — vì $S = 2 \cdot \text{hop}$ nên tổng cửa sổ = 1 tại mọi điểm, khớp nối không méo.

- Sau dual-path: masker = Linear($N, 2N$) $\rightarrow$ sigmoid 2 mặt nạ $\rightarrow$ est_i =

mask_i·E $\rightarrow$ Decoder (ConvTranspose1d $k=16, s=8$) $\rightarrow$ (B, 2, 1, T). Encoder: Conv1d($1 \rightarrow 256$,

k=16, s=8) + chuẩn hóa toàn cục embedding (Sepformer-style).

4.5 `models/f0\_conditioning.py` — F0 → embedding (điều kiện hóa thanh điệu)

Lý thuyết. Thanh điệu tiếng Việt nằm ở *đường nét* F0, không phải cao độ tuyệt đối → mỗi khung tạo 3 đặc trưng: $\log(F0)$, cờ vô thanh, và $\log(F0) - \text{median}(\log F0 \text{ voiced})$ (chuẩn hóa theo người nói → giữ contour, bỏ identity). F0 của mixture (2 người) không định nghĩa chặt → dùng cao độ trội.

Code:

```
voiced = (f0 > 1.0).float()
log_f0 = torch.log(f0.clamp_min(1e-3))
med = median của các khung voiced (mỗi câu) # chuẩn hóa người nói
feats = stack([log_f0, voiced, log_f0 - log(med)], -1)
return self.net(feats) # MLP 3→32→32→f0_dim → (B, T, f0_dim)
```

resample_f0: nội suy tuyến tính riêng giá trị F0 và mặt nạ voiced; khung đầu ra chỉ voiced nếu mặt nạ nội suy > 0.5 → không "lên" F0 sang vùng vô thanh. Sau đó embedding này đi vào F0ConditionedSSM._modulate (4.1 (4)) — cổng tanh $\text{Linear}(f0_dim \rightarrow 2 \cdot d_state)$ tạo thang nhân cho B, C từng lớp.

4.6 `models/positional.py` — PE hình sin

$pe[:, 0::2] = \sin(pos \cdot div)$, $pe[:, 1::2] = \cos(pos \cdot div)$ (Vaswani) — cộng vào chiều S và C của dual-path (4.4 (2)) để bù thứ tự cho Mamba.

4.7 `utils/metrics.py` — SI-SNR, PIT và các chỉ số cải thiện

Lý thuyết (SI-SNR). Với ước lượng $\hat{s}$ chiếu lên tham chiếu s :

```
s_target = ((s^s) / ||s||^2) · s ; e_noise = s^ - s_target
SI-SNR = 10 · log10( ||s_target||^2 / ||e_noise||^2 )
```

Tách 2 người chưa biết thứ tự → **PIT**: thử 2 hoán vị, lấy hoán vị tối ưu. $SI-SNR_i = SI-SNR(est, ref) - SI-SNR(mix, ref)$ (cải thiện so với tín hiệu trộn).

Code:

```
def si_snr(estimate, reference): # đúng công thức trên, vector (B,)
def si_snr_pit(estimates, references): # ma trận (B,2,2) rồi max(identity, swap)
def si_snr_loss(...): return -best.mean() # train: âm = tốt hơn
def si_snri(est, ref, mixture): return mean_i[ si_snr(est_perm_i, ref_i)
                                              - si_snr(mix, ref_i) ]
```

$si_snri(mix-as-estimate) = 0$ (sanity test đã chạy); hoán vị đúng/đảo cho cùng kết quả nhờ PIT (đã test). PESQ/STOI tính ở 16 kHz qua pesq/ pystoi (bọc try/except → NaN nếu thiếu gói). $me_asure_latency$ đo ms + VRAM.

4.8 `data/vivos.py` — dynamic mixing + F0 proxy

SpeechSeparationDataset.__getitem__: chọn câu 2 từ **người nói khác nhau** (qua `_speaker_id` x), RMS bằng nhau, trộn SNR ~ $U[0,5]$ dB (make_mixture), cắt/pad về dur_sec; F0 hỗn hợp = mix_f0(max) của 2 contour nguồn (proxy — val/test dùng F0 mixture thật). split_speakers chia theo người nói (không trùng fold). Bản CVS (train/speech_separation/dataloader/) đọc scp mix s1 s2 [f0] + cắt segment ngẫu nhiên — cùng triết lý, định dạng khác.

4.9 `preprocess/` — F0 và mixture

- extract_f0.py: PyWorld dio (ước lượng F0) → stonemask (làm mượt) →

f0.npy float32, 10 ms, 0 = vô thanh; cache để huấn luyện không trả chi phí online (100 fps khớp hằng số F0_FPS trong dataloader).

- make_mixtures.py: cặp khác người nói cố định (seed), sinh val/test một lần;

F0 trích từ mixture thật — đúng thứ mô hình thấy lúc suy luận.

4.10 `train.py` — phương pháp huấn luyện

Thành phần	Code	Lý thuyết
Seed	`set_seed(42)` (`utils/seed.py`)	tái lập kết quả
Optimizer 2 nhóm	`build_optimizer`: nhóm tham số thuộc `F0ConditionedSSM`/`Mamba2Core` có `lr×0.5`	**Mamba nhay LR** → học chậm hơn phần Conv (R1)

Schedule	`WarmupCosineSchedule`: `lr=(step+1)/warmup` rồi cosine về `min_lr`	Transformer convention; warmup ổn định SSM
Loss	`si_snr_loss` (PIT, âm = tốt)	mục 4.7
AMP	`torch.autocast("cuda")` + `GradScaler`; grad clip 5.0	fp16 tiết kiệm VRAM; clip chống nổ
Vòng lặp	epoch → accum → optimizer.step → `scheduler.step()` → val mỗi `eval_every_epochs` → early-stop (patience 10) → checkpoint `{model,optimizer,epoch,step,config}`	ổn định + resume
Checkpoint	`runs/<run>/checkpoints/{best,last}.pt`, `--resume`	mất điện/gián đoạn không mất tiến trình

4.11 Bản ClearerVoice-Studio (`train/speech\_separation/`) — khác gì?

Cùng model (packaging HybridBiMamba_SS(args).model), framework của MOSSFormer2: checkpoint **con trỏ** last_checkpoint/last_best_checkpoint (ghi tên file model.ckpt-{epoch}-{step}.pt) thay vì đường dẫn cố định; **LR halving x0.5 mỗi 5 epoch không cải thiện** (không warmup — Adam lr 1.5e-4); batch 1 + grad accumulation effec_batch_size; dataloader cắt segment max_length giây; scp thêm cột F0 tùy chọn; display/TensorBoard giữ nguyên CVS. Xem README.md trong thư mục đó. So sánh tham số 2 bản: parameter_analysis.md.

4.12 `evaluate.py` / `inference.py`

evaluate.py = FixedMixtureDataset(test) → si_snr/sdri + compute_perceptual (PESQ/STOI) + measure_latency (GPU/CPU, xuất JSON). inference.py = đọc wav → extract_f0_from_wav (nếu use_f0) → model → ghi _s1.wav/_s2.wav; không có pyworld thì chạy không-F0 kèm cảnh báo.

5. Bảng thuật ngữ (Lý thuyết ↔ Ký hiệu code)

Lý thuyết	Code	Ghi chú
d_model	`encoder_dim` / `d_model`	256 mặc định
d_inner = expand·d	`self.d_inner`, `expand=2`	512
d_state (N)	`d_state=64`	độ rộng trạng thái h
Δ (delta)	`dt_proj` + `delta_bias` + `softplus`	bước thời gian
A (chuyển trạng thái)	`A_log` → `A = -exp(A_log)`	buffer, không train
B, C (input/output)	`x_proj` → `Bmat, Cmat` (+ F0 gate)	chính là điểm chèn F0
z (gated output)	`z` từ `in_proj`, nhân `SiLU(z)`	như Mamba
Cửa sổ cục bộ	`d_conv=4` (SSM), DWConv k=31 (khối)	2 mức cục bộ khác nhau
Intra / inter	`intra_blocks` / `inter_blocks`	trong đoạn / liên đoạn
PIT SI-SNR	`si_snr_pit`	hoán vị 2 người nói
F0 contour	`f0.npy` (100 fps) → `F0Projector` → gate	10 ms/khung, 0 = vô thanh

6. Checklist đọc nhanh theo file

- models/ssm.py → F0ConditionedSSM.forward + selective_scan_ref (lỗi Mamba).
- models/bimamba.py → BiMambaLayer.forward (2 chiều).
- models/hybrid_block.py → HybridBlock vs ConformerBlock (phép thay thế).
- models/separation.py → DualPathProcessor.forward (dual-path) → HybridSepformer.
- models/f0_conditioning.py → F0Projector + resample_f0.
- utils/metrics.py → si_snr_pit / si_snr_i (loss & chỉ số).
- train.py → build_model / build_optimizer / vòng epoch (huấn luyện).
- train/speech_separation/models/hybrid_bimamba/hybrid_bimamba.py →

HybridBiMamba_SS (bản MOSSFormer2-style, cùng lỗi).

Toàn bộ số liệu (tham số/MACs/thời gian) trong các tài liệu khác được đo từ code này — xem

cách tái tạo ở cuối `parameter\_analysis.md`.

Hybrid Conformer-BiMamba — Hướng dẫn đọc Code (Lý thuyết ↔ Code)

Nguồn: docs/code_walkthrough_vi.md · tạo tự động từ Markdown